

Topic 2.2

2.2: Function Prototypes

A rule of C and C++ is that identifiers must be declared before they may be used. On the surface, this forces us to define all our functions before the main program.

Writing programs in this manner is cumbersome, because we see all the small functions first before the main program, that is, we see too many details prior to seeing the big picture. It also hinders code reusability.

For this reason, and some other important ones, we can use *prototypes*. A prototype is a declaration of a function (*not* a definition) that gives the compiler enough information about a function so that the compiler knows how the function is supposed to be invoked. This allows the compiler to verify that all invocations of the function are reasonable.

For example:

```
void print_results( float init_odom,  
                  float final_odom,  
                  float gas, float mpg );
```

is a prototype. Notice the `;` as the end of the prototype. We are allowed to omit the formal parameter names if we wish:

```
print_results( float, float, float, float );
```

is an equivalent prototype. After seeing either prototype, the compiler knows

- the name of the function;
- the function's return type; and
- the number and type of its formal parameters.

Because the compiler knows about this function now, we can use it in our code before the actual code of the function itself appears.

Function Signatures

In C++, we may have multiple versions of a function with the same name, multiple versions of an operator, etc. Thus the compiler must be able to distinguish which particular version of a function or operator we intend.

This is achieved through the use of function signatures. Each function must have a unique signature. This signature consists of

- the name of the function;
- the number of formal parameters; and
- the types of the formal parameters.

Topic 2.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-3>)